

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 14 : Communication Sans Fil

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

14.0 Introduction : Couper le cordon

L'ère de l'IoT (Internet of Things)

Un capteur isolé a une utilité limitée. La véritable puissance de l'électronique embarquée réside dans sa capacité à transmettre ses données à distance et à recevoir des commandes sans contrainte matérielle.

Dans ce chapitre, nous allons apprendre à :

- Utiliser des modules Radio Fréquence (RF) 433 MHz à très bas coût.
- Établir une liaison série sans fil via Bluetooth (Modules HC-05 / HC-06).
- Contrôler l'Arduino depuis une application Smartphone.
- Découvrir la norme 802.15.4 (ZigBee) et les modules XBee.
- Structurer des données pour les envoyer fiablement dans les airs.

14.1 Les Modules RF 433 MHz (Low-Cost)

Le choix de l'économie : Vendus par paires (un émetteur et un récepteur distincts), ces petits modules coûtent moins de 2 euros.

Caractéristiques :

- **Fréquence :** 433 MHz (Bande libre ISM - Industrielle, Scientifique et Médicale).
- **Portée :** De 10 mètres à l'intérieur jusqu'à 100 mètres en extérieur (si une antenne filaire de 17.3 cm est soudée).
- **Unidirectionnel (Simplex) :** L'émetteur ne fait qu'envoyer, le récepteur ne fait qu'écouter. Pas de confirmation de réception matérielle (ACK).
- **Sensibilité au bruit :** Très sensibles aux parasites électromagnétiques.

14.2 La Bibliothèque RadioHead (ex-VirtualWire)

Le problème des ondes radio "nues" : Si vous branchez simplement la broche TX (Série) sur l'émetteur 433 MHz, les bits vont se perdre dans le bruit de fond constant des ondes radio.

La solution : L'encapsulation La bibliothèque RadioHead (spécifiquement la classe RH_ASK) ou l'ancienne VirtualWire crée un protocole robuste :

1. Envoie un signal de réveil (Préambule).
2. Ajoute la longueur du message.
3. Envoie les données.
4. Ajoute une somme de contrôle (CRC) pour vérifier si le message a été corrompu en vol.

Code : Émetteur RF 433 MHz

Câblage de l'émetteur : VCC sur 5V, GND sur GND, et DATA sur la broche 12 (par défaut pour RadioHead).

```
#include <RH_ASK.h>
#include <SPI.h> // Requis par RadioHead meme sans l'utiliser

RH_ASK driver; // Utilise la broche 12 pour l'emission par defaut

void setup() {
  Serial.begin(9600);
  if (!driver.init()) {
    Serial.println("Erreur init Radio !");
  }
}

void loop() {
  const char *msg = "Alerte Intrusion !";

  // Envoi du message (doit etre converti en tableau d'octets)
  driver.send((uint8_t *)msg, strlen(msg));
  driver.waitPacketSent(); // Attend que le message soit entierement parti

  Serial.println("Message envoye.");
}
```

Code : Récepteur RF 433 MHz

Câblage du récepteur : VCC sur 5V, GND sur GND, et DATA sur la broche 11.

```
#include <RH_ASK.h>
#include <SPI.h>

RH_ASK driver; // Utilise la broche 11 pour la reception par default

void setup() {
  Serial.begin(9600);
  driver.init();
}

void loop() {
  uint8_t buf[RH_ASK_MAX_MESSAGE_LEN];
  uint8_t buflen = sizeof(buf);

  // Verifie si un message est arrive et est valide (CRC correct)
  if (driver.recv(buf, &buflen)) {
    Serial.print("Message reçu : ");

    // Affiche chaque caractere
    for (int i = 0; i < buflen; i++) {
      Serial.print((char)buf[i]);
    }
  }
}
```

14.3 Le Bluetooth (Classic)

Pour une communication de proximité sécurisée et bidirectionnelle avec des smartphones ou des PC, le Bluetooth est le standard industriel.

Les modules HC-05 et HC-06 :

- **HC-06** : Fonctionne uniquement en mode Esclave (Slave). Il attend qu'un smartphone se connecte à lui.
- **HC-05** : Peut être Maître ou Esclave. Il peut se connecter automatiquement à un autre HC-05.
- **Fonctionnement** : Ces modules créent un "pont" sans fil invisible. Ce que vous envoyez sur la broche série de l'Arduino sort sur le Bluetooth du téléphone (profil SPP - Serial Port Profile).

14.4 Câblage et Sécurité Électrique du HC-05

Le module Bluetooth communique via les broches TX et RX. Pour garder la broche USB libre pour le débogage (sur Uno), on utilise `SoftwareSerial` sur les broches 10 et 11.

Le Piège de la tension (Rappel du Chap 13)

Le module s'alimente en 5V, mais **ses broches logiques (RX) ne tolèrent que 3.3V !** Il faut absolument utiliser un diviseur de tension (ex : $1k\Omega$ et $2k\Omega$) entre le TX de l'Arduino et le RX du module Bluetooth.

[Insérer le schéma de câblage HC-05 avec pont diviseur de tension sur la broche RX]

Code : Interface Bluetooth bidirectionnelle

Ce code crée un pont : ce que vous tapez sur le PC (Moniteur Série) part sur le Bluetooth, et vice-versa.

```
#include <SoftwareSerial.h>

// Broche 10 = RX_Arduino (Connecte au TX du HC-05)
// Broche 11 = TX_Arduino (Connecte au RX du HC-05 via diviseur)
SoftwareSerial BTSerial(10, 11);

void setup() {
  Serial.begin(9600);    // Vers le PC via USB
  BTSerial.begin(9600);  // Vers le HC-05 (Vitesse par défaut)
  Serial.println("Pret a communiquer !");
}

void loop() {
  // Lit depuis le Bluetooth -> Ecrit sur le PC
  if (BTSerial.available()) {
    Serial.write(BTSerial.read());
  }
  // Lit depuis le PC -> Ecrit sur le Bluetooth
  if (Serial.available()) {
    BTSerial.write(Serial.read());
  }
}
```

14.5 Contrôler un projet depuis un Smartphone

Étape 1 : Le Jumelage (Pairing) Allez dans les paramètres Bluetooth de votre téléphone Android, cherchez le "HC-05" et entrez le code PIN (généralement 1234 ou 0000).

Étape 2 : L'Application Téléchargez une application comme **Serial Bluetooth Terminal** sur le Play Store.

Étape 3 : La Logique Arduino Dans le code Arduino, vous pouvez lire les caractères reçus :
`if(charRecu == 'A') { allumerLaLampe(); }`

14.6 Introduction à XBee (La norme ZigBee)

Les limites du 433 MHz et du Bluetooth : Portée limitée, pas de véritable réseau, collisions si plusieurs émetteurs parlent en même temps.

La solution industrielle : XBee Les modules XBee (fabriqués par Digi International) utilisent la norme radio 802.15.4.

- **Fréquence :** 2.4 GHz (Mondialement libre).
- **Portée :** De 30 mètres (série 1 standard) à plus de **1 Kilomètre** (série Pro).
- **Réseau Maillé (Mesh) :** Un module XBee peut faire "rebondir" le message d'un capteur lointain pour qu'il atteigne la base (Routage automatique).

14.7 Configuration préalable des modules XBee

Contrairement aux modules simples, les XBee sont de véritables petits ordinateurs. Il faut les configurer **avant** de les brancher à l'Arduino.

On utilise pour cela un adaptateur USB ("XBee Explorer") et le logiciel gratuit **XCTU**.

Paramètres vitaux (Mode Transparent) :

- **PAN ID (Personal Area Network) :** L'identifiant de votre réseau. Seuls les XBee avec le même PAN ID peuvent se parler.
- **DH / DL (Destination Address) :** L'adresse de destination (à qui on envoie).
- **MY (16-bit Source Address) :** L'adresse de ce module lui-même.

14.8 Câblage d'un module XBee

Format physique propriétaire

Les broches d'un XBee ont un espacement de 2mm (et non 2.54mm comme l'Arduino). Il ne rentre **pas** dans une Breadboard. Il faut un "XBee Shield" ou un adaptateur (Breakout board).

Câblage (Mode Transparent) :

- **VCC** : Strictement 3.3V ! (Certains shields font la conversion depuis le 5V).
- **GND** : Masse.
- **DIN (RX)** : Connecté au TX de l'Arduino.
- **DOUT (TX)** : Connecté au RX de l'Arduino.

Code : Communication XBee (Mode Transparent)

En "Mode Transparent" (aussi appelé AT mode), le XBee se comporte exactement comme un long câble série invisible. Le code est identique à celui du Bluetooth !

```
// Avec deux Arduino equipes chacun d'un XBee correctement configure :

void setup() {
  Serial.begin(9600); // Demarre la communication avec le XBee
}

void loop() {
  // L'Arduino 1 envoie :
  Serial.println("Valeur Capteur : 42");
  delay(1000);

  // L'Arduino 2 lira ce meme message en utilisant :
  // if (Serial.available()) { String msg = Serial.readString(); }
}
```

14.9 Pour aller plus loin : Le Mode API XBee

Les limites du Mode Transparent : Si vous avez 5 capteurs qui envoient des données à un XBee central, comment savoir qui a envoyé quoi en mode transparent ?

Le Mode API (Application Programming Interface) : Dans ce mode, l'Arduino n'envoie plus du simple texte, mais construit des paquets de données (Frames) précis contenant :

- L'adresse MAC exacte du destinataire.
- Le type de paquet.
- Les données (Payload).

Le mode API est complexe à coder manuellement. On utilise la bibliothèque tierce `xbee-arduino` de `xbee.h` pour générer et décoder ces paquets matériels.

Résumé du Chapitre 14

- **Radio 433 MHz** : Solution idéale pour l'apprentissage et les petits budgets, à condition d'utiliser la bibliothèque `RadioHead` pour fiabiliser la transmission.
- **Bluetooth (HC-05)** : Parfait pour remplacer le câble USB et créer des interfaces de contrôle depuis un smartphone Android. Attention aux niveaux logiques (3.3V).
- **XBee** : La solution industrielle pour les longues portées et les réseaux de capteurs multiples. Nécessite une configuration matérielle préalable (XCTU).
- **La règle universelle** : Toute communication sans fil est asynchrone et sujette aux interférences. Prévoyez toujours dans votre code des mécanismes pour gérer la perte d'un message !

Fin du Chapitre 14. Le Chapitre 15 conclura la partie communication en connectant l'Arduino au monde via Internet (Ethernet et Réseau).